

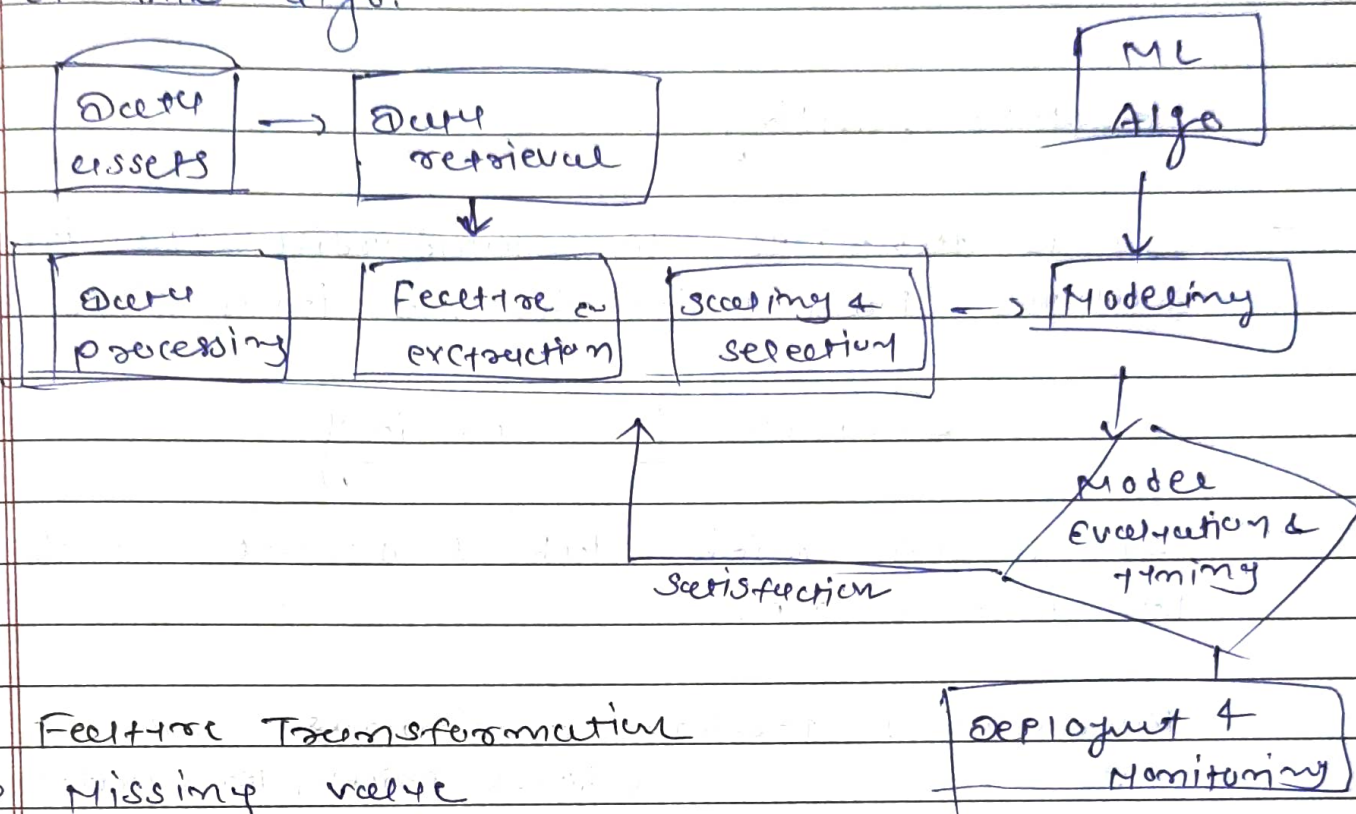
Feature Engineering

classmate

Date _____

Page _____

→ It is the process of using domain knowledge to extract features from raw data. these features can be used to improve the performance of ml algo.



1) Feature Transformation

- Missing value
- Handling categorical
- outliers detection
- Feature scaling

2) Feature Construction

→ in titanic data sets 2 columns

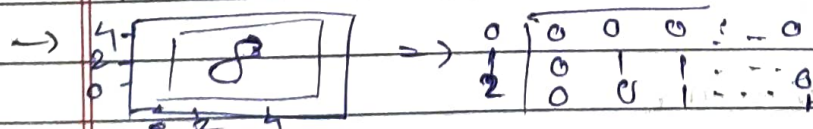
- 1) sibsp, sibling, spouse } family
- 2) parch - parents, child } family

0 - Alone

0-4 - small family

4 < n - Large "

3) Feature Selection



MNIST datasets.

→ remove unnecessary or - pixels from image

4) Feature Extraction

→ Reduce feature

→ Algo: PCA, LDA, TSME

* Feature Scaling

⇒ It is technique to standardize the independent features present in the data in a fixed range.

Supervised Learning

→ It works with labeled datasets. each input has known o/p.

Regression: predicts continuous values
(house prices, temperature)Classification: predict discrete categories.
(spam detection, disease diagnosis)Linear Regression→ It is supervised ML algo used to predict the ^{value of} relation b/w dependent variable and 1 or more independent variable.

$$y = mx + b$$

y = dependent variable

x = independent "

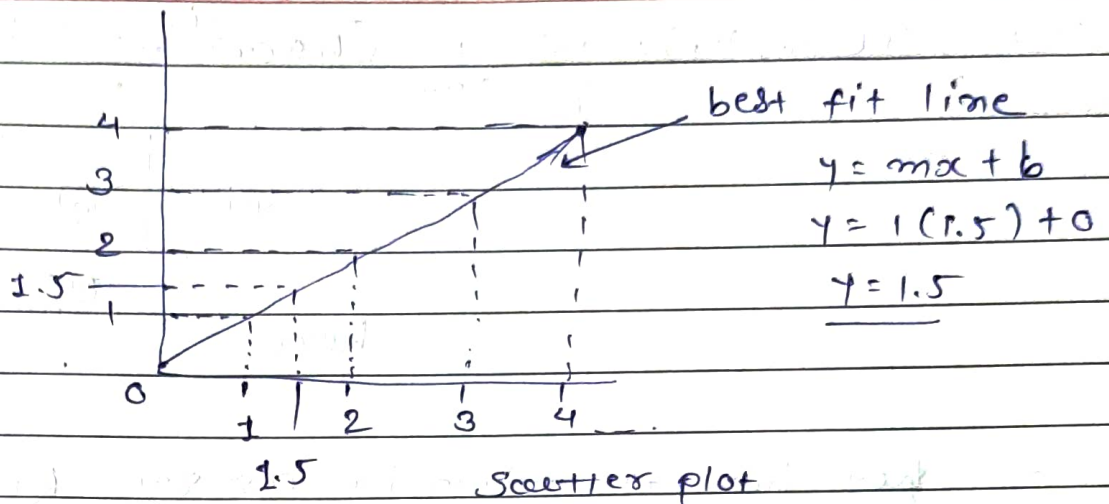
m = slope

b = intercept

ex:

$$m = 1$$

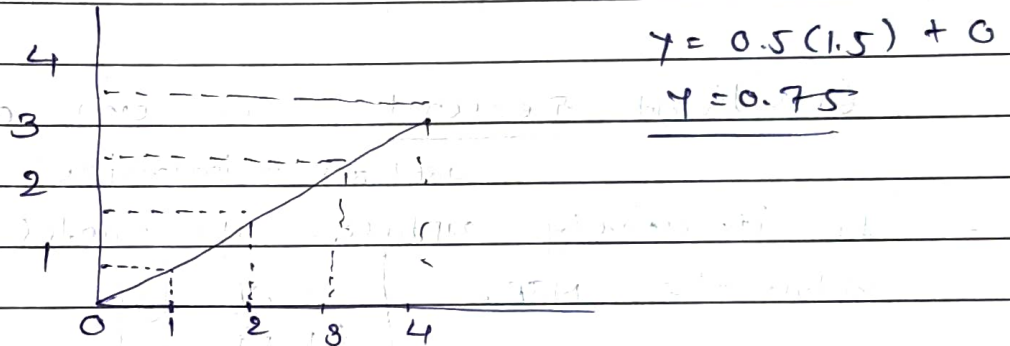
$$b = 0$$



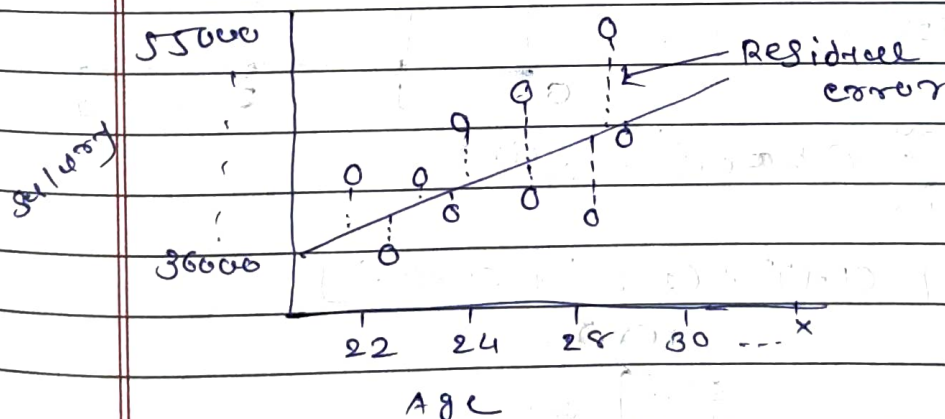
ex:

$$m = 0.5$$

$$b = 0$$



Residual error : represent difference b/w actual val & predicted val from the model.



Cost Function : It is the Mean Squared Error (MSE), which calculates the avg of the squared errors b/w the predicted & actual values.

→ Goal is to minimize this error and to find the best-fit line.

$$h_{\theta} = \theta_0 + \theta_1 x$$

$$y = mb + mx$$

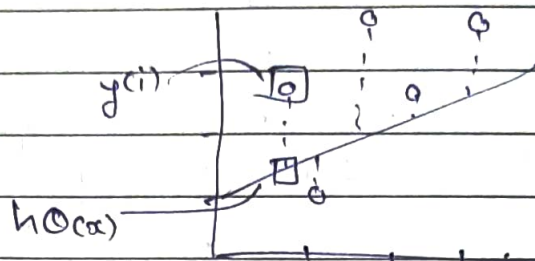
classmate

Date _____

Page _____

cost sum

$$J(\theta_0, \theta_1) = \sum_{i=1}^m \frac{1}{2m} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



→ we have to minimize cost J by changing the value of θ_0, θ_1 .

Gradient Descent : is an optimization algo used to minimize the cost function.

→ It iteratively updates the model parameters to reduce the MSE.

1) $\theta_0 = 0, \theta_1 = 1$

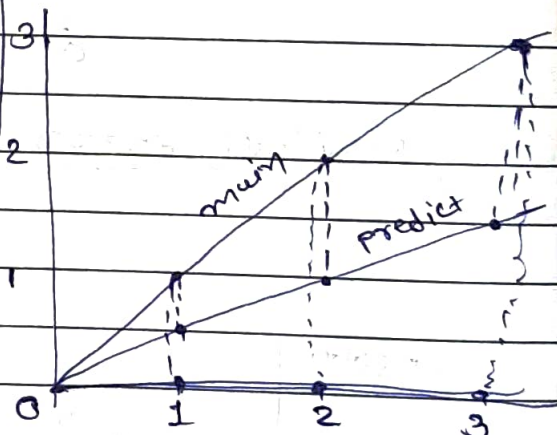
$$h_{\theta} = \theta_0 + \theta_1 x$$

$$= 0 + 1(1) = 1$$

$$= 0 + 1(2) = 2$$

$$= 0 + 1(3) = 3$$

x	y
1	1
2	2
3	3

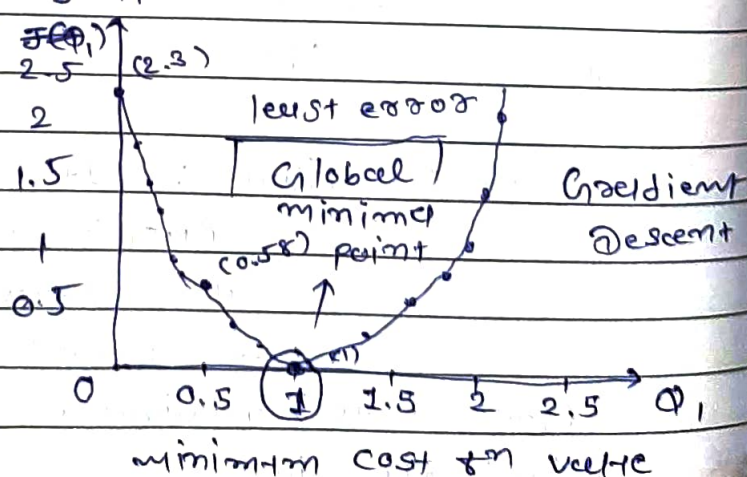


$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$= \frac{1}{6} [(1-1)^2 + (2-2)^2 + (3-3)^2]$$

$$J(\theta_1) = 0$$

$$J(\theta_1) = 0$$



2)

$$\phi_1 = 0.5, \phi_0 = 0$$

$$h(\phi) = \phi_0 + \phi_1(x)$$

$$= 0 + 0.5(1) = 0.5$$

$$= 0 + 0.5(2) = 1.0$$

$$= 0 + 0.5(3) = 1.5$$

$$J(\phi) = \frac{1}{6} [(1-0.5)^2 + (2-1)^2 + (3-1.5)^2]$$

$$J(\phi) = 0.58$$

3)

$$\phi_1 = 0, \phi_0 = 0$$

$$h(\phi) = 0 + 0(1) = 0$$

$$= 0 + 0(2) = 0$$

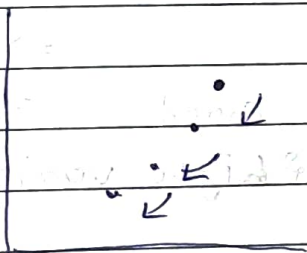
$$= 0 + 0(3) = 0$$

$$J(\phi) = \frac{1}{6} [(1-0)^2 + (2-0)^2 + (3-0)^2]$$

$$= 2.3$$

Repeat Convergence Theorem

(Hyperparameters)



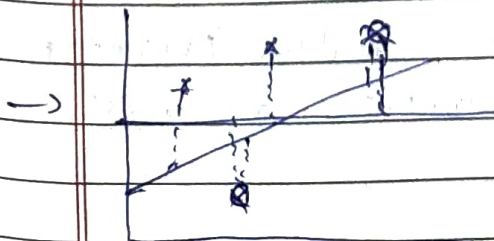
Learning Rate (α): size of steps taken during the optimization process to minimize the loss function.

Q: Why you use learning rate as a low value whenever you are making gradient descent?

→ It allows the model to make smaller updates, which can help prevent overshooting the minimum loss and getting stuck in local minima.

Performance Matrix

$$R^2 = 1 - \frac{\text{Sum of residuals}}{\text{Sum of total}} \Rightarrow 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2}$$



The result will be always positive and in decimal.

ex: $0.86 \approx 86\%$

Age $\rightarrow$ salary $\Rightarrow 87\%$ Good

location $\rightarrow$ salary $\Rightarrow 89\%$ Good

sleep $\Rightarrow 91\%$ bad

$$\text{Adjusted } R^2 = \frac{1 - (1 - R^2)(n - 1)}{N - P - 1}$$

N = no of rows
 P = features

Q Overfitting & Underfitting

$\rightarrow$ Overfitting

{low bias}

Training - Model perform better $\Rightarrow 92\%$

Testing - Model Not perform Good $\Rightarrow 62\%$

{high variance}

Underfitting :

{high bias}

Training - Model perform bad $\Rightarrow 68\%$

Testing - Model Not perform good $\Rightarrow 64\%$

{low variance}

Perfect

Model :

Training - 92% {low bias}

Testing - 90% {low variance}

Q Regularization

$\rightarrow$ It is set of technique used to reduce/prevent overfitting by adding penalize on model complexity.

- 1) Lasso (L1) : Add penalty equal to the absolute value of model

$$L1 = \underbrace{\frac{1}{2m} \sum_{i=1}^m [h(x^{(i)}) - y^{(i)}]^2}_{\text{MSE}} + \lambda \sum_{i=1}^m |slope|$$

- 2) Ridge (L2) : Add penalty equal to the square of the models

$$L2 = \text{MSE} + \lambda \sum_{i=1}^m (slope)^2$$

ML-3

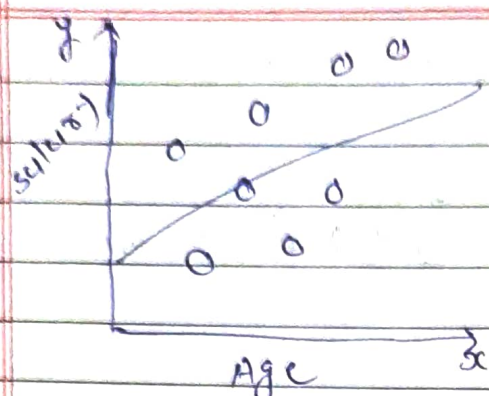
- | | | | |
|---|---------------------|--------------------------------|--------|
| 1. classification
Algo | 3. KNN | 5. Random Forest
classifier | 7. SVM |
| 2. Logistic
Regression | 4. Decision
Tree | 6. Naive Bayes | |
| 9. K-Fold cross
validation | | | |
| 8. Evaluation Metrics
Accuracy, Precision, Recall
F1 score, confusion Metrics | | 10. Hurry on project | |

1) Classification

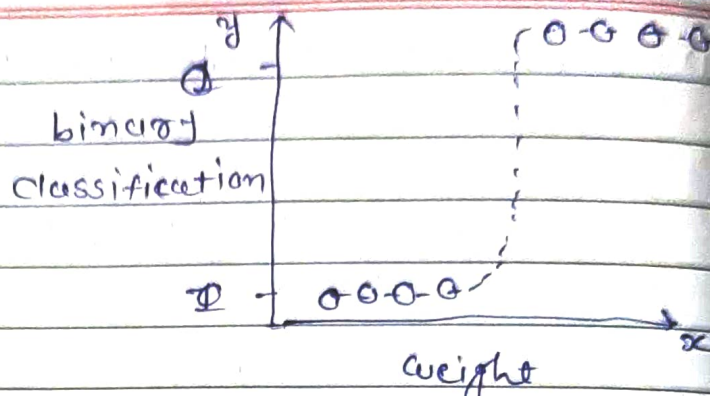
→ It's algo used to predict category or class label for given input.

ex: Is this cat or dog?

will this customer survive or not?

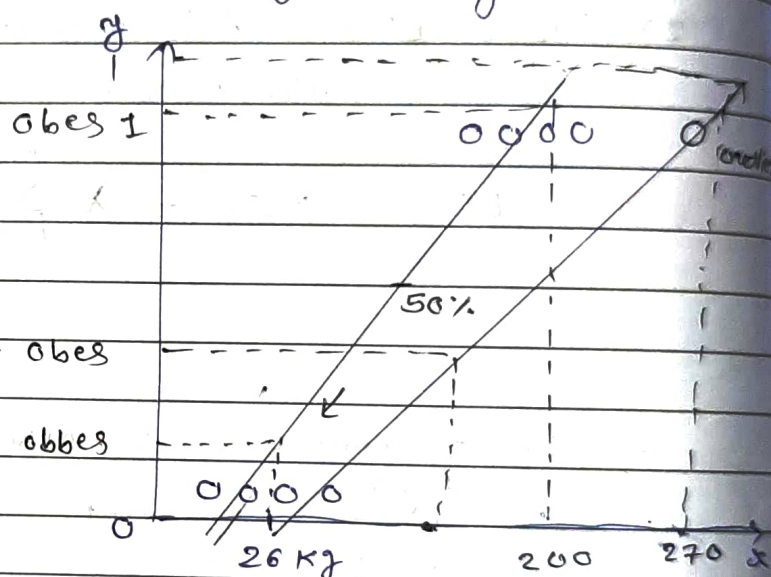


Linear Reg.



Logistic Reg

Weight	obesity
92	yes
63	No
24	No
38	No
102	yes
190	yes



- We don't draw straight line in logistic reg.:
- 1) Outliers - Influence the straight line
 - 2) Result can be negative or and positive $\pm$

→ To prevent all of these things we squash the line.

→ To make the straight into 'S' shaped curve we will use SIGMOID activation fn.

↓
→ It can map any real valued number into a range b/w 0 & 1.

۷

$$g = \frac{1}{1 + e^{-2}}$$

Line converted into
'S' shaped.

$$= 2.71828$$

→ gives smooth curve ex: $/ \rightarrow \sqcup \rightarrow \cup$

Logistic Reg.

$\alpha_0, \alpha_1 \rightarrow$ values $\rightarrow$ To create best fit line

best squash line

→ cost function $\rightarrow$ Log loss function.

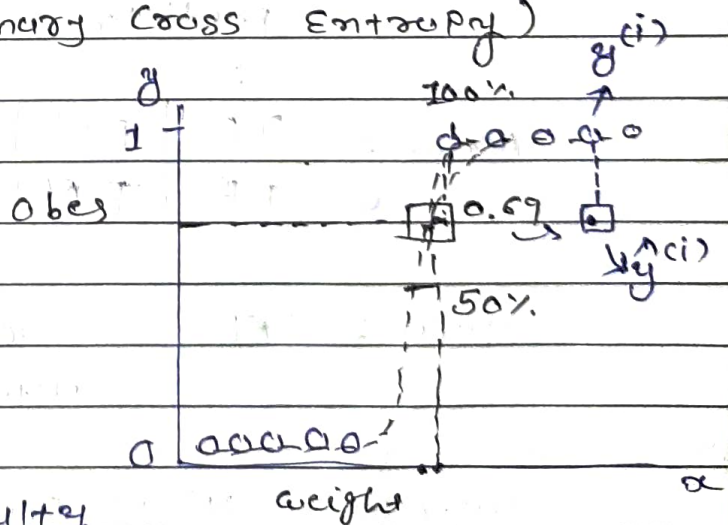
=> Log loss function (Binary Cross Entropy)

1) if predicted prob = 0.99 &
actual = 1
 $\Rightarrow$ very less loss

2)	if predicted prob = 0.01
	& actual = 1

$\Rightarrow$ very high loss $\Rightarrow$ Penalty

↳ It calculate with log loss & η



$$\Rightarrow \log \text{loss} = \frac{-1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$$

classmate
Date _____
Page _____

m = number of sample

$y^{(i)}$ = actual label (0, 1)

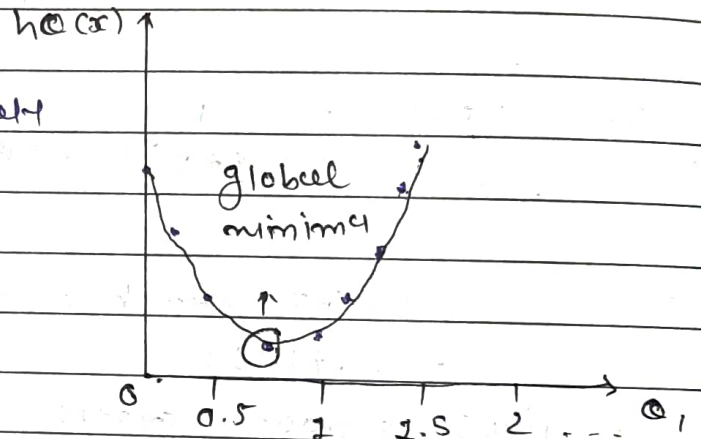
$\hat{y}^{(i)}$ = Predicted probability

$$y^i = 1$$

$$\hat{y}^i = 0.54$$

$$\log \text{loss} = 0.616$$

$\theta_0, \theta_1 \Rightarrow$ Perfect value



AS Evaluation of Model

* Confusion Matrix

→ It is a table used to check how well a classification model works.

		Predicted		
		1	0	250
Actual	1	TP	FN	→
	0	FP	TN	
		190	10	
		10	40	

Type-1 (FP) $\Rightarrow$ Detecting something that is not actually present.

Type-2 (FN) $\Rightarrow$ Failing to detect something is actually present.

1) Accuracy : How many total predictions correct.

$$= \frac{TP + TN}{TP + TN + FP + FN}$$

- 2) Precision: Out of all predicted positive, How many are actually positive

$$\Rightarrow \frac{TP}{TP + FP}$$

- Use when FP are costly (Spam del)
 ex: Model says "Spam" 10 times and only 7 spam
 $= 7/10 = 70\%$

- 3) Recall: Out of all actually positive, how many did we correctly predicted

$$\Rightarrow \frac{TP}{TP + FN}$$

- Use when FN are costly (Disease detection)

- 4) F1 score: Harmonic mean of precision & Recall

$$F1 \text{ score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Use when you want balance b/w precision & Recall

KNN (K-Nearest Neighbors)

- Finding nearest points, used for both classification & Regression tasks.
 → It is non-parametric & instance based learning (No assumption about data distribution)

Internet Typing Speed Wickets

24

40

1

12

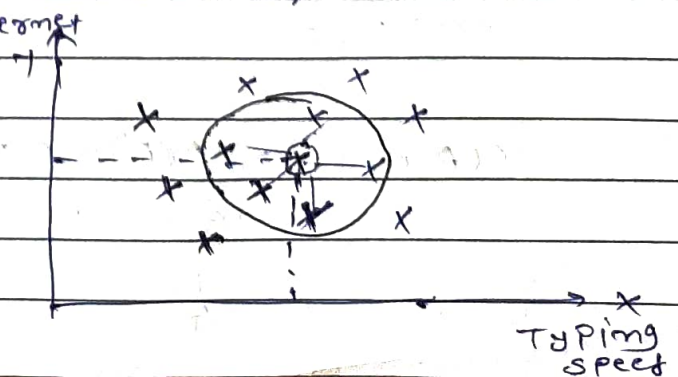
22

0

⋮

⋮

⋮



K values

- 1) Define K values $\Rightarrow K=5$ (hyperparameter)
- 2) calculate distance from all points (Euclidean dist)
- 3) Sort all distance $\rightarrow$ 3-yes, 2-NO
- 4) Majority count
 $\rightarrow$ yes person is hacker

$\rightarrow$ Other than classification $\rightarrow$ Mean of 5 nearest neighbors

$\Rightarrow$ K values must be odd $\rightarrow$ experiment $\rightarrow$ 2000
 1600 train 400 Test

$$K=5 \Rightarrow 86\%$$

$$K=3 \Rightarrow 89\%$$

$$K=25 \Rightarrow 93\% \uparrow$$

Naive Bayes

$\rightarrow$ Find the probability of an event based on new information or evidence.

Independent

Coin $\Rightarrow P(H) = 1/2, P(T) = 1/2$

Dice $\Rightarrow P(6) = 1/6$
 $P(4) = 1/6$

$$P(A \text{ and } B) = P(A) \times P(B)$$

$$= 1/2 \times 1/2$$

$$= 1/4$$

Dependent

Deck of cards $\sqrt{52}$

$$P(R) = \frac{26}{52} = \frac{1}{2} \quad R(26) \quad B(26)$$

$$P(R) = \frac{25}{51} = \frac{1}{2}$$

$$P(A \text{ and } B) = P(A) \times P(B/A)$$

$$P(A \text{ and } B) = P(B \text{ and } A)$$

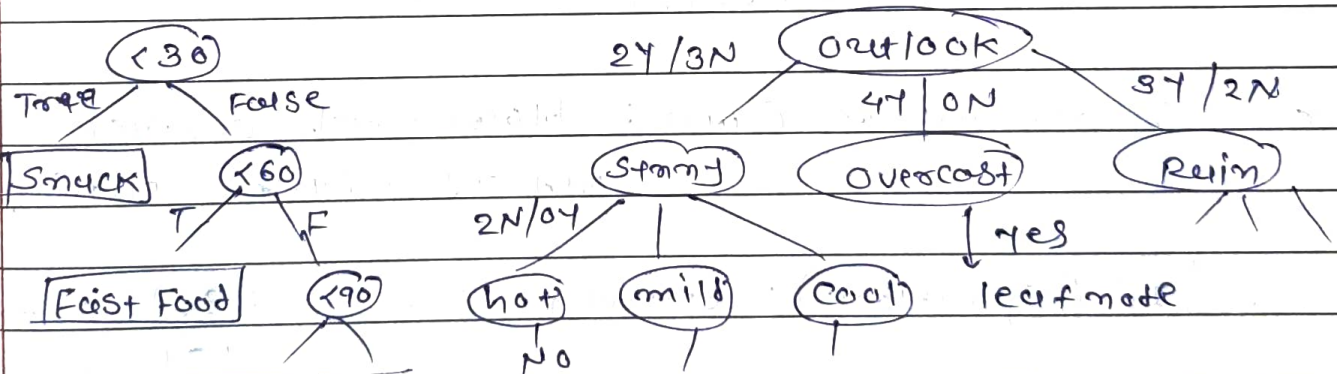
$$P(A) \times P(B/A) = P(B) \times P(A/B)$$

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)}$$

Algo

Decision Tree

- It is supervised learning used for both classification & regression.
- It has a hierarchical tree structure which consists of root node, branches, internal node & leaf node.



Flow works :

- Chooses best features
- Split data in groups
- Repeat again & again
- Stop when : 1> Prediction clear
2> max depth is reached

* Entropy: How much confusion is in data?
high entropy = more confusion
low " = less "

$$S = -P_+ \log_2(P_+) - P_- \log_2(P_-)$$

ex: play = 5 NO, 9 yes

$$S = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right)$$

$$S = 0.940$$

Outlook 2Y/3N

$$\begin{aligned} \text{ Sunny} &\Rightarrow -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) = 0.971 \\ \text{ Overcast} &\Rightarrow 0 \\ \text{ Rain} &\Rightarrow 0.971 \end{aligned}$$

* Information Gain: How much confusion is reduced after splitting?

$$\Rightarrow IG(S, \text{outlook}) = \text{Entropy}(S) - \sum \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$= 0.940 - \left[\frac{5}{14} \times 0.975 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.971 \right]$$

$$\text{Outlook} = \boxed{0.246 \text{ IG}} \rightarrow \text{high}$$

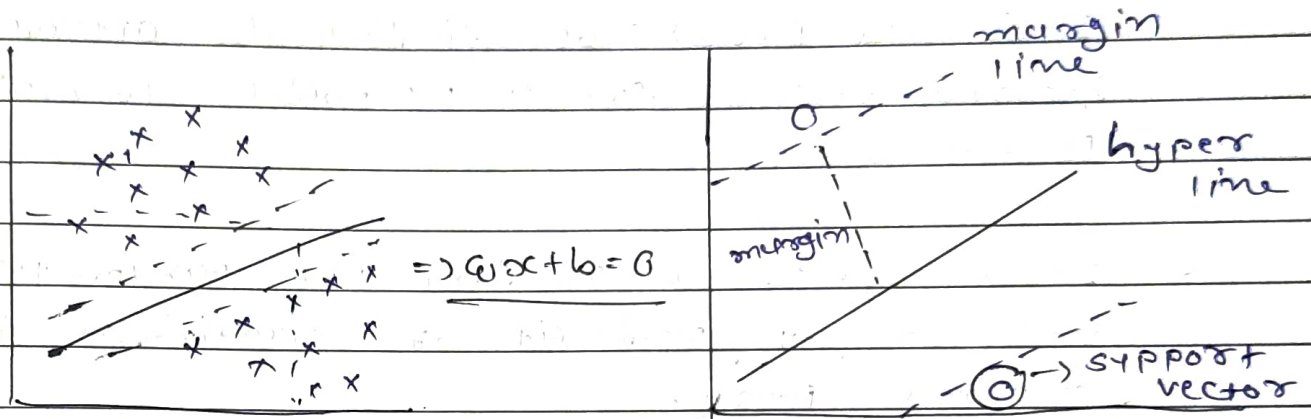
$$\text{Temperature} = \sim 0.029$$

$$\text{Humidity} = \sim 0.191$$

$$\text{Wind} = \sim 0.048$$

A SVM (Support Vector Machine)

- The goal of SVM is to find the optimal hyperplane that separates data points of different classes in a N -dimensional space.

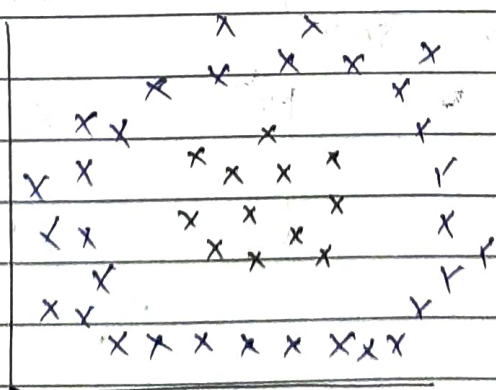


x = point you want to predict

w = weight

b = bias

- Hyperplane: A line that separates 2 categories
- Margin: The space b/w the line & nearest point
- Support vector: The closest base to margin that support the line.



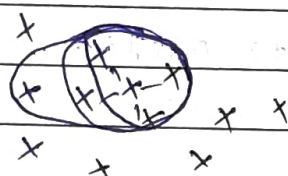
⇒ We use kernel to distinguish blue & black points

⇒ A kernel is mathematical ϕ that transform the input data into a higher-dimensional space to make it easier to classify.

Model Tuning

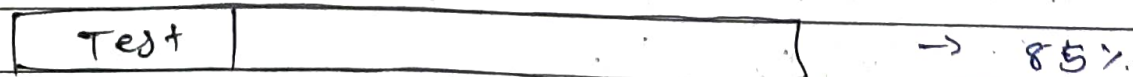
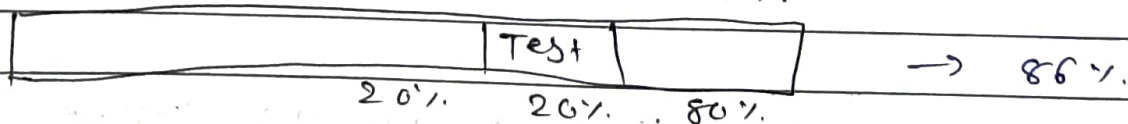
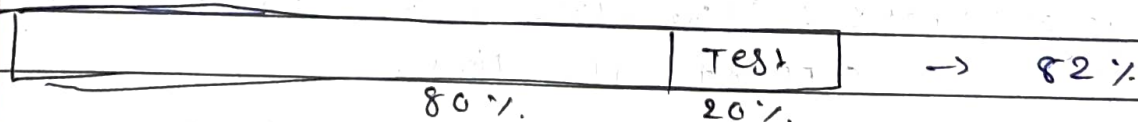
pt-4.

- When we train a model, it has certain settings these are called hyperparameters.
- It is the process of finding the optimal set of hyperparameters for a ML models to maximize its performance.
- If we just use random hyperparameters, our model might underfit or overfit, or simply perform poorly.



$K=3, \quad K=5$
 $1Y, 2N \quad 3Y, 2N$

Cross Validation : Used to check how well a ML model perform on unseen data while preventing overfitting.



Avg = 83%

- Divide data in K -parts and every part testing data remaining will be your training data.

Hyperparameter Tuning

- Ridge & Lasso regression (α) = Learning rate
- KNN → $n_neighbors = [3, 5, 7, 9]$
- Decision Tree
 - ↳ max-depth
 - ↳ Sample split
 - ↳ max features
- SVM ⇒ kernel()
- Random forest
- DB Scan

Methods

- 1) Manual Search
- 2) Grid search CV
- 3) Randomized search

Grid Search CV

- Exhaustive search method: Grid search systematically tries every possible combination of hyperparameter you specify.

Ex: Testing every recipe variation until you find the tastiest one.

Ex:

KNN

$n = [3, 5, 7, 9, 11]$

weight = ['Uniform', 'distance']

metric = ['manhattan', 'euclidean']

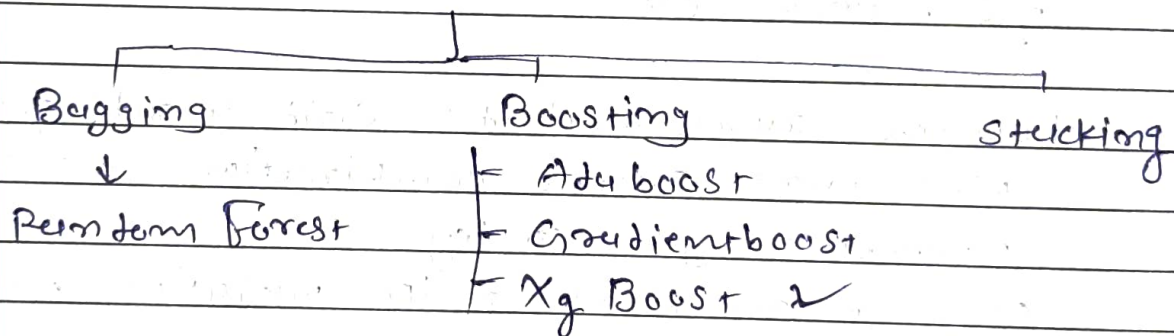
N	weight	metric	Acc	Experiment
3	Uniform	manhattan	⇒ 9	↓ Avg Acc
3	distance	11	⇒ 9	
3	Uniform	euclidean	⇒ 9	
3	distance	11	⇒ 9	
			⇒	

2) Random Search CV (niter = 10)

→ It is a hyperparameter optimization method that samples a fixed no of parameter combination, multi from specified distributions, making it more efficient than exhaustive grid search in cross-validation.

Ensemble Learning

→ It is powerful technique in ML that combine the predictions of multiple models to improve accuracy and robustness.



1) Stacking

→ Is an ensemble learning technique that combines the predictions of multiple base models to create a stronger predictive model.

Architecture : 2 layers to improve model accuracy

1) Base Model (M1-0) : That directly learn from the original training data.

→ DT, RF, LR. etc.

(LR or LA)

2) Meta-Model (M1-1) : combine the base model predictions in smart way to make final pm.



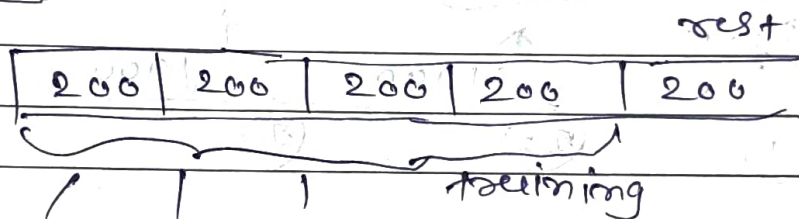
Prediction at each model are given as input feature for meta classifier

Meta-Model

Final Prediction

EX:

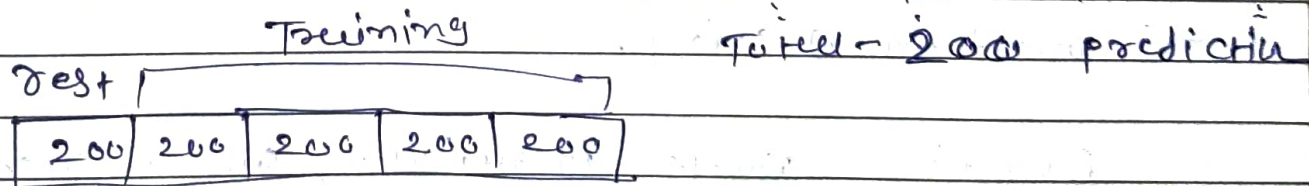
→ 1000 points



LR SVM DT → Base Models

Meta-Model - KNN ⇒ SVM-pred DT-pred LR-pred KNN

1)	0	1	1
	0	1	0
	0	0	1
	1	1	1



2) 20% | 80%
SVM DT LR ⇒ again 200 prediction

Meta Model = 200 + 200 = 400

↳
Total = 1000 prediction

low bias
high variance
overfitting

low bias
low variance
Generalized

classmate

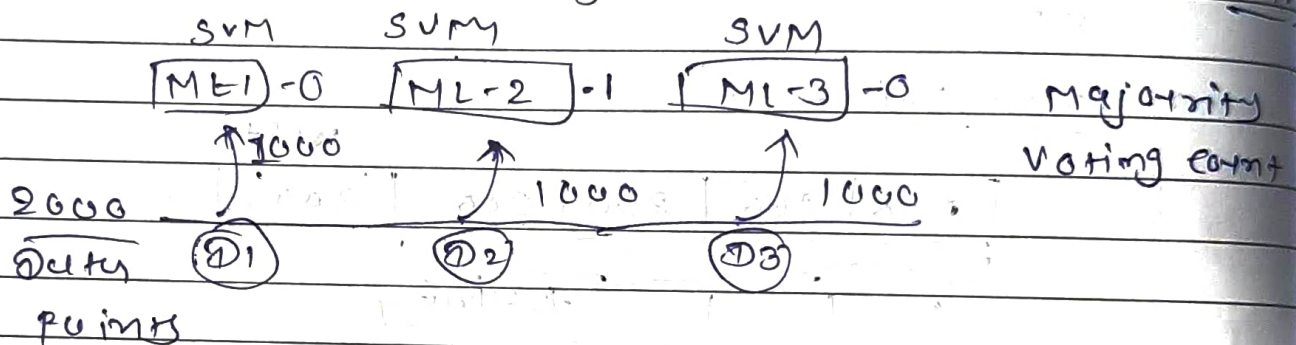
Date

Page

2) Bugging

→ Short for Bootstrap Aggregating, design to improve accuracy and stability of ML models by reducing variance.

→ Train model independently in parallel, it is well-suited for high-variance model like DT.



3) Boosting

→ It design to improve accuracy of predictive models by combining multiple weak learners into single strong learner. weak learner are model that perform slightly better than random guessing.

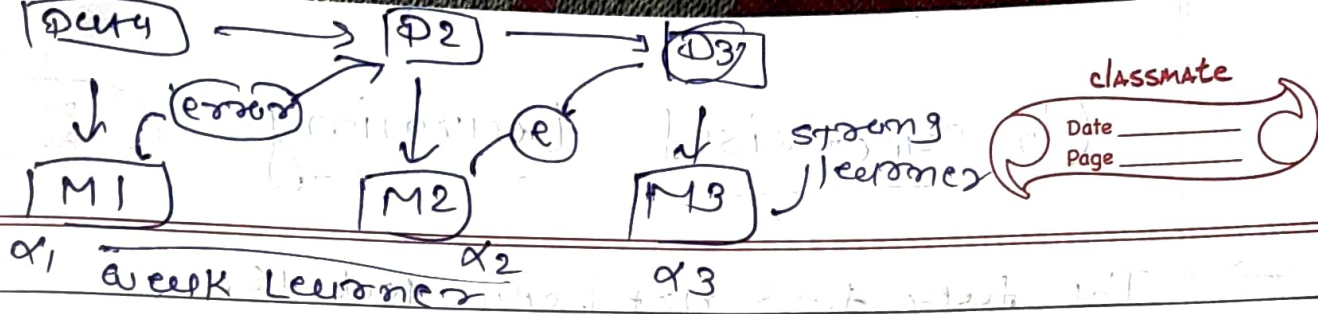
→ Boosting work iteratively, where each new model focuses on correcting the error of its predecessor, thereby enhancing overall perform.

high bias
high variance

underfitting

low bias
low variance

Generalized



$$b = \alpha_1(m_1) + \alpha_2(m_2) + \alpha_n(m_n)$$

Learning rate Model (boost)

3 Types:

- 1) Ada Boost
- 2) Gradient Boost
- 3) XGB Boost (Extreme Gradient Boost)

Unsupervised Learning

classmate

Date _____
Page _____

- The data does not have labels.
- The model must find pattern, structure & relationships, in the data by itself.

Goal:

- Find hidden structure in data
- Group similar data points
- Reduce data dimension while keeping important info (Dimensionality Reduction)
- Detect outliers / anomalies
- ex: DB scan, clustering, Anomaly Detection

Common Technique:

- clustering - grouping similar data point
- Dimensionality Reduction - reducing no of features (PCA)
- Anomaly Detection - finding outliers

Use cases:

- Customer Segmentation - group customer by behavior Spending
- Market 11 → group of market or products
- Anomaly Detection → Fraud, network security
- Recommendation Systems
- Image Compression → reduce img size
- Medical Data → disease pattern
- preprocessing → use PCA to improve supervised models

K-Means Clustering

- It is algorithm used to group data points into n -clusters based on their similarity.
- The random-start parameter

1. Decide n clusters ($K=4$)
2. Initialize centroids
3. Assign cluster → Euclidian Distance
4. Re-assign centroids
5. Finish

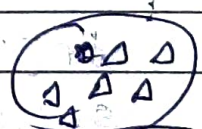
⇒ For making the cluster & deciding K -value, you have to use elbow method.

Elbow Method :

- The elbow methods help select a best no. of clusters by plotting the Within-cluster sum of squares (WCSS) or Sum of Squared Errors (SSE) againsts different values of K and looking for point where the rate of improvement sharply decreases, forming an "Elbow" on graph.

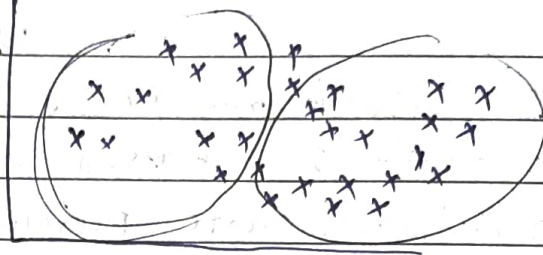
Disadvantage :

- 1) we have to initialize K -value
- 2) outliers will move the center points



$\left\{ \begin{array}{l} K\text{-Means} \rightarrow \text{Centroid based} \\ DB \rightarrow \text{Density based} \end{array} \right.$

3) Not good perform on circular data



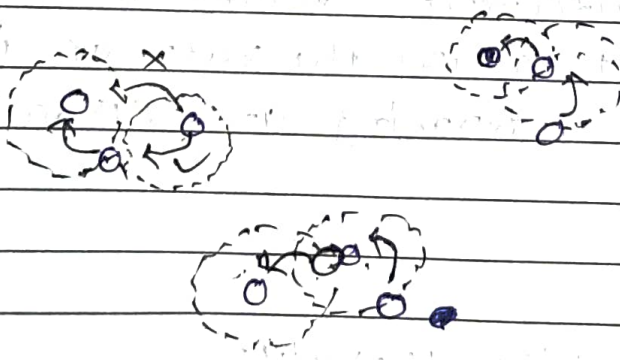
2) DB Scan (Density based spatial clustering of Application with noise)

- Mainly used for arbitrary-shaped cluster/Noise
- Non-parametric Algo
- Epsilon Distance

Key parameter:

- 1) Epsilon (ϵ): Define radius of the neighborhood (units) around point
- 2) Minpts: Minimum no of points required

ex:



```
cat > dbscan = DBSCAN(eps = 0.4, min_samples = 5)
db_label = dbscan.fit_transform(x_scaled)
```

```
df['dbscan-cluster'] = db_label
```

```
sns.scatterplot(x='F1', y='F2', hue='db-cluster',
                data=df)
```


③ Dimensionality Reduction

- Process of reducing the no of input variable or features in dataset while preserving its essential info and structure.
- High dimensional datasets with many features can be sparse & computationally expensive to analyze, often leading to the "curse of dimensionality" where model perform poorly.
- Very difficult to find pattern
- KNN, K-Means, DBSCAN → distance based algo
→ Fail in high dimension
- Storage
- Model training time ↑
- 150 D → 50 D

1) Feature Selection

- It chooses most relevant feature from the datasets.
- (1) Filter method
- (2) Wrapper methods
- (3) Embedded methods
- (4) Forward Feature Selection
- (5) Backward Feature Elimination

2) Feature Extraction

- creating new features by combining or transforming the original features.

(1) PCA : Convert correlated variable into uncorrelated variable called "principal components".

(2) LDA : Linear Discriminate Analysis

(3) ICA : Independent Component Analysis

Real world use :

→ Text categorization

→ Image Retrieval

→ Gene expression analysis

Ex :

